7.1. Parte 1: inventario de la colección

El programa deberá generar automáticamente una tabla resumen con, al menos:

Fichero	MIME	Codificación	Idioma	Caracteres extraídos
...	...	...	...	...

Compare el tipo detectado por Tika con la extensión del fichero. Incluya deliberadamente **un fichero cuya extensión haya sido modificada** y explique el resultado obtenido.

7.2. Parte 2: contenido, metadatos y enlaces

Para cada documento deberá extraerse:

- texto;
- metadatos disponibles;
- idioma;
- enlaces, cuando existan.

Seleccione al menos tres formatos diferentes y compare cualitativamente la información obtenida. No se pide enumerar todos los metadatos, sino identificar cuáles podrían resultar útiles para un sistema de RI y cuáles parecen poco fiables o dependientes del formato.

7.3. Parte 3: documentos compuestos

Procese al menos un recurso que contenga otros documentos (por ejemplo, un ZIP o un formato con recursos embebidos). Identifique el documento principal y los recursos internos que Tika sea capaz de descubrir.

Explique qué estrategia utilizaría para indexarlos:

- como un único documento;
- como documentos independientes;
- como documentos independientes relacionados con el contenedor.

Justifique la decisión en función de un escenario de búsqueda concreto.

7.4. Parte 4: representación normalizada

El programa generará un fichero `coleccion.jsonl` con un registro por documento procesado. Como mínimo se incluirán los siguientes campos:

```
id, path, mime_type, language, title, author,
text, num_characters, links, status
```

Pueden añadirse otros campos que se consideren relevantes. Los errores de procesamiento no deberán detener la colección completa: el registro correspondiente deberá indicar el error mediante el campo `status` o mediante información adicional.

7.5. Parte 5: términos y ley de Zipf

A partir del texto extraído de uno de los documentos de tamaño suficiente, obtenga las frecuencias de sus términos. Como mínimo deberá realizarse conversión a minúsculas y una tokenización razonable; cualquier tratamiento adicional deberá explicarse.

Genere un CSV ordenado por frecuencia decreciente:

```
Termino;Frecuencia
de;124
el;103
...
```

Represente frecuencia frente a rango y el correspondiente gráfico log–log (para ello se puede utilizar cualquier hoja de cálculo). Analice si la distribución presenta aproximadamente el comportamiento descrito por la ley de Zipf:

$$F(R) \approx \frac{k}{R^m}$$

y estime k y m mediante un ajuste de potencia o regresión lineal si consideramos logaritmos sobre

$$\log F = \log k - m \log R.$$

No es necesario que el ajuste sea perfecto. Interesa interpretar qué significa la fuerte desigualdad en las frecuencias de los términos para la construcción de un índice y para operaciones posteriores como la eliminación de palabras vacías o la ponderación de términos.

Este análisis lo deberemos hacer con documentos suficientemente grandes (o concatenando varios documentos) que representen las probabilidades de un idioma. También se debe considerar documentos con distintos idiomas (inglés, francés, castellano). En Prado podemos encontrar un fichero ejemplo llamado frecuenciaPlateroyYo.csv.

7.6. Parte 6: del documento al buscador

Considere ahora que `coleccion.jsonl` debe utilizarse en dos sistemas distintos:

a) un buscador tradicional basado en un **índice invertido**;

b) un sistema RAG que segmenta los documentos y obtiene **embeddings** de los fragmentos.

Incluya en la memoria una breve discusión respondiendo a las siguientes cuestiones:

- ¿Qué campos conservaría para cada sistema y cuáles indexaría?
- ¿Qué metadatos permitirían aplicar filtros durante la recuperación?
- ¿Qué consecuencias tendría una extracción incorrecta del texto en cada sistema?
- ¿Cómo trataría un documento del que no se ha podido extraer texto?
- En el sistema RAG, ¿qué información de procedencia conservaría para poder identificar el documento original a partir de un fragmento recuperado?

No es necesario implementar el índice invertido ni el sistema RAG. Esta parte pretende relacionar la etapa de procesamiento documental construida en la práctica con las siguientes etapas de un sistema real de RI.

8. Aspectos que se valorarán

Se valorarán especialmente:

- corrección y robustez de la implementación;
- uso adecuado de Tika y de sus mecanismos de detección y parsing;
- tratamiento correcto de errores y documentos problemáticos;
- calidad de la representación JSONL generada;
- análisis crítico de los resultados, no sólo la presentación de salidas;
- capacidad para relacionar la extracción documental con conceptos de Recuperación de Información.

No se valorará positivamente añadir complejidad innecesaria al programa. El objetivo es construir una etapa de procesamiento documental clara, reproducible y razonablemente robusta.